



Relatório Testes de Software

Plataforma GREENHERB — Gestão Inteligente de Estufa

Unidade Curricular: Engenharia de Software II

Ano Letivo: 2025/2026

G-PL1-Extra3

João Silva – 17405

Davide Cardoso – 28445

Data de Entrega: 5/6/2026

Relatório Testes de Software

Plataforma GREENHERB — Gestão Inteligente de Estufa

Licenciatura Engenharia Informática

Unidade Curricular: Engenharia de Software II

Ano Letivo: 2025/2026

G-PL1-Extra3

João Silva – 17405

Davide Cardoso – 28445

Data de Entrega: 5/6/2026

Resumo

O presente relatório documenta o trabalho de testes de software desenvolvido sobre a plataforma GREENHERB — um sistema de gestão inteligente de estufa de ervas aromáticas, exposto através de uma API REST em Node.js. O sistema sob teste (SUT) cobre onze recursos (/auth, /users, /herbs, /plans, /batches, /tasks, /measurements, /alerts, /automation, /reports e /audit) com autenticação JWT e três perfis de utilizador.

Foram implementados três níveis de teste: (1) testes de unidade com isolamento total por mocks e stubs (Jest), validando individualmente funções como authenticate(), validatePlan(), importHerbsFromCsv() e decideAction(); (2) testes de integração com Supertest sobre a stack completa, cobrindo todos os endpoints da API; e (3) testes de sistema executando fluxos end-to-end completos.

As técnicas formais aplicadas foram: Particionamento de Equivalência (PE) para identificação de classes válidas e inválidas de entrada; Análise de Valores Limite (BVA) nos parâmetros numéricos com intervalos definidos; Cobertura de Condições Múltiplas (MCC) com tabelas de verdade para todos os 36 ifs da função validatePlan(); e MC/DC para a função decideAction() do motor de automação. O trabalho foi organizado em seis sprints, produzindo aproximadamente 499 casos de teste no total.

Palavras-chave:

Testes de Software, GREENHERB, Caixa-Preta, Caixa-Branca, Particionamento de Equivalência, Análise de Valores Limite, MC/DC, Cobertura de Condições Múltiplas, Testes de Integração, Duplos de Teste, Jest, Supertest.

Índice

1. Identificação do Grupo	4
2. Descrição do Sistema Sob Teste (SUT)	4
2.1 Recursos da API	4
2.2 Perfis de Utilizador	5
3. Conceitos Fundamentais de Testes de Software	5
3.1 Tipos de Teste	5
3.2 Técnicas de Caixa-Preta (Black Box)	7
3.3 Técnicas de Caixa-Branca (White Box)	8
3.4 Duplos de Teste	9
4. Estratégia de Testes Adotada	10
5. Plano de Testes Detalhado por Sprint	11
5.1 Sprint 1 — Autenticação	11
5.2 Sprint 2 — Importação CSV e Criação de Planos	12
5.3 Sprint 3 — Requisitos Restantes	14
5.4 Sprint 4 — Testes de Integração	16
5.5 Sprint 5 — Testes White-box (MCC)	18
5.6 Sprint 6 — Duplos de Teste (Stub e Mock)	20
5.7 Testes de Sistema	22
6. Particionamento de Equivalência	23
7. Análise de Valores Limite (BVA)	25
8. Cobertura de Condições Múltiplas (MCC e MC/DC)	26
9. Matriz de Rastreabilidade	28
10. Resultados de Execução	28
11. Conclusões, Limitações e Propostas de Melhoria	29
Anexo A — Matriz de Rastreabilidade Completa	31
Anexo B — Diagrama de Classes Global (Sprint 6)	31

Lista de Tabelas

<i>Tabela 1 — Recursos da API GREENHERB</i>	<i>4</i>
<i>Tabela 2 — Perfis de Utilizador</i>	<i>5</i>
<i>Tabela 3 — Duplos de Teste</i>	<i>9</i>
<i>Tabela 4 — Níveis de Teste e Ferramentas</i>	<i>10</i>
<i>Tabela 5 — Classes de Equivalência: username</i>	<i>11</i>
<i>Tabela 6 — Classes de Equivalência: password</i>	<i>11</i>
<i>Tabela 7 — Middleware de Autenticação</i>	<i>12</i>
<i>Tabela 8 — Classes de Equivalência: conteúdo CSV</i>	<i>13</i>
<i>Tabela 9 — BVA: parâmetros de planos</i>	<i>13</i>
<i>Tabela 10 — Ficheiros e contagens Sprint 3</i>	<i>14</i>
<i>Tabela 11 — BVA: Justificação de Alertas</i>	<i>14</i>
<i>Tabela 12 — MC/DC: decideAction()</i>	<i>15</i>
<i>Tabela 13 — Cobertura de Endpoints (Sprint 4)</i>	<i>16</i>
<i>Tabela 14 — Resumo de decisões MCC validatePlan()</i>	<i>18</i>
<i>Tabela 15 — IF-01: Tabela de Verdade</i>	<i>18</i>
<i>Tabela 16 — IF-02: Tabela de Verdade</i>	<i>19</i>
<i>Tabela 17 — IF-06: Tabela de Verdade</i>	<i>19</i>
<i>Tabela 18 — IF-09: Tabela de Verdade</i>	<i>19</i>
<i>Tabela 19 — Correspondência Teste → IF (extracto)</i>	<i>20</i>
<i>Tabela 20 — Stub vs. Mock: decisão de projecto</i>	<i>21</i>
<i>Tabela 21 — Casos de Teste SensorGateway Stub</i>	<i>21</i>
<i>Tabela 22 — Casos de Teste NotificationGateway Mock</i>	<i>22</i>
<i>Tabela 23 — Testes de Sistema</i>	<i>22</i>
<i>Tabela 24 — PE: Tipo de Plano</i>	<i>23</i>
<i>Tabela 25 — PE: Classificação de Alertas</i>	<i>23</i>
<i>Tabela 26 — PE: Estado de Lote</i>	<i>24</i>
<i>Tabela 27 — PE: Perfil de Utilizador</i>	<i>24</i>
<i>Tabela 28 — PE: Decisão sobre Alertas</i>	<i>24</i>
<i>Tabela 29 — BVA: Temperatura</i>	<i>25</i>
<i>Tabela 30 — BVA: Humidade Relativa</i>	<i>25</i>
<i>Tabela 31 — BVA: Luminosidade</i>	<i>25</i>

Tabela 32 — BVA: Duração do Ciclo 26

Tabela 33 — BVA: Justificação para Ignorar Alerta 26

Tabela 34 — Resumo MCC Sprint 5 27

Tabela 35 — Rastreabilidade Inversa (extracto) 28

Tabela 36 — Métricas de Cobertura 29

1. Identificação do Grupo

1. Identificação do Grupo

Tabela 1 — Identificação do Grupo

<i>Campo</i>	<i>Informação</i>
<i>Grupo</i>	<i>G-PL1-Extra3</i>
<i>Elementos</i>	<i>ESTGV17405-João Silva ; 28445-Davide Cardoso +2 elementos?</i>
<i>Repositório</i>	<i>https://github.com/MarcioLourenco5/ES2/tree/main/PROJETO_FINALV2</i>
<i>Instruções de execução</i>	<i><code>npm install → npm test (Jest) + npx jest --coverage --runInBand</code></i>
<i>Framework de testes</i>	<i>Jest + Supertest (Node.js)</i>

2. Descrição do Sistema Sob Teste (SUT)

A plataforma GREENHERB é um sistema de gestão inteligente de estufa de ervas aromáticas. O Sistema Sob Teste (SUT) é a sua API REST, desenvolvida em Node.js, que expõe os recursos necessários ao planejamento de cultivos, monitorização ambiental, geração de alertas, automação por regras e auditoria de operações.

A API segue a especificação OpenAPI 3.x (documentada via Swagger) e assenta em autenticação por JWT, com expiração e renovação de tokens. A persistência é feita sobre uma base de dados relacional, sendo que os testes utilizam uma camada de dados em memória (mockData) para garantir independência de SGBD.

2.1 Recursos da API

Tabela 2 — Recursos da API GREENHERB

<i>Recurso</i>	<i>Descrição</i>
<i>/auth</i>	<i>Autenticação, emissão e renovação de tokens JWT</i>
<i>/users</i>	<i>Gestão de utilizadores (perfis: Técnico, Responsável, Administrador)</i>
<i>/herbs</i>	<i>Catálogo de ervas aromáticas, com suporte a importação CSV/Excel</i>
<i>/plans</i>	<i>Planos de cultivo (regular, emergência, pontual)</i>

<i>/batches</i>	<i>Lotes de cultivo com estado, divisões, perdas e produtividade</i>
<i>/tasks</i>	<i>Tarefas operacionais (rega, fertilização, colheita, monitorização)</i>
<i>/measurements</i>	<i>Medições ambientais de temperatura, humidade e luminosidade</i>
<i>/alerts</i>	<i>Alertas Informativos, Avisos e Críticos, com resolução/ignorar justificado</i>
<i>/automation</i>	<i>Regras de automação e comutação Manual/Automático</i>
<i>/reports</i>	<i>Exportação de relatórios em CSV ou Excel</i>
<i>/audit</i>	<i>Logs de auditoria de operações relevantes</i>

2.2 Perfis de Utilizador

- Administrador — acesso total; gere utilizadores e importa catálogos.
- Responsável Técnico — cria e autoriza planos; resolve alertas.
- Técnico — regista medições, executa tarefas, abre e fecha lotes.

3. Conceitos Fundamentais de Testes de Software

Esta secção apresenta uma descrição concisa de cada tipo de teste e de cada técnica utilizada no trabalho, servindo de referência para as secções seguintes.

3.1 Tipos de Teste

Testes de Unidade

Um teste de unidade valida o comportamento de uma única função, método ou classe de forma completamente isolada — sem base de dados, sem rede, sem ficheiros. Sempre que a unidade tem dependências externas (JWT, bcrypt, sensores, notificações), estas são substituídas por duplos de teste (mocks, stubs, fakes) que simulam o comportamento esperado sem executar código real. O foco é a lógica de domínio: o que a função faz com as entradas que recebe.

Quando usar: em funções de validação, cálculo, classificação e regras de negócio — tudo o que pode ser exercitado sem iniciar o servidor.

Vantagens: muito rápidos de executar; falhas apontam exactamente para a linha de código errada; não dependem de estado externo.

Exemplos neste projecto: `validatePlan()`, `authenticate()`, `decideAction()`, `importHerbsFromCsv()`.

Testes de Integração

Um teste de integração verifica se múltiplas componentes colaboram correctamente quando ligadas entre si. Em vez de isolar uma função, a stack completa (rotas → middleware de autenticação → controlador → serviço → dados) é exercitada com pedidos HTTP reais. O objectivo é confirmar

que os contratos da API são cumpridos: códigos de estado correctos, estrutura JSON esperada, autenticação validada, efeitos secundários produzidos.

Quando usar: para testar endpoints completos da API, garantindo que o fluxo de dados entre camadas funciona sem rupturas.

Vantagens: detectam erros de integração que testes de unidade não podem encontrar (ex: middleware que bloqueia um pedido válido, ou serviço que não persiste correctamente).

Exemplos neste projecto: POST /plans com tipo inválido, POST /measurements que gera alerta, PATCH /alerts/:id com justificação no limite.

Testes de Sistema

Um teste de sistema avalia a aplicação como um todo, executando fluxos end-to-end completos em condições o mais próximas possível da produção. Não se testa uma função nem um endpoint em isolamento — testa-se um cenário de negócio completo, do início ao fim, como um utilizador real o faria.

Quando usar: para validar fluxos críticos que atravessam múltiplos recursos da API (ex: criar erva → criar plano → abrir lote → registar medições → fechar lote → verificar produtividade).

Vantagens: garantem que o sistema funciona como um produto coerente; detectam falhas de integração de alto nível que testes unitários e de integração isolados não cobrem.

Exemplos neste projecto: ciclo completo de um lote (TS-01), gestão de incidente com plano de emergência (TS-02), controlo de acesso ao plano pontual (TS-03), auditoria de operações (TS-04).

3.2 Técnicas de Caixa-Preta (Black Box)

Nas técnicas de caixa-preta, o testador concebe os casos de teste com base no comportamento especificado (requisitos, documentação da API) — sem conhecer nem analisar o código interno. O sistema é tratado como uma caixa opaca: dadas certas entradas, esperam-se certas saídas.

Particionamento de Equivalência (PE)

O Particionamento de Equivalência parte do princípio de que valores dentro de uma mesma classe de entrada se comportam todos da mesma forma. Em vez de testar exaustivamente todos os valores possíveis (impossível), identificam-se classes de equivalência — grupos de valores que o sistema trata de forma idêntica — e testa-se apenas um representante de cada classe.

As classes dividem-se em:

- Válidas — valores que o sistema deve aceitar e processar correctamente.
- Inválidas — valores que o sistema deve rejeitar com um erro específico.

Exemplo neste projecto: para o campo type de um plano, as classes válidas são "regular", "emergência" e "pontual"; as classes inválidas incluem qualquer outra string, null, undefined e tipos não-string (número, booleano).

Intenção: reduzir o número de testes sem sacrificar cobertura — um único teste por classe é suficiente para representar todos os valores equivalentes.

Análise de Valores Limite (BVA — Boundary Value Analysis)

A Análise de Valores Limite complementa o PE focando-se nos pontos de fronteira dos intervalos numéricos, onde os erros de programação são mais frequentes (operadores $<$ vs $<=$, off-by-one). Para cada intervalo aceite $[min, max]$, testam-se cinco valores canónicos:

Tabela 3 — Cinco valores canónicos BVA

<i>Posição</i>	<i>Valor</i>	<i>Descrição</i>
<i>min - 1</i>	<i>Abaixo do limite inferior</i>	<i>Deve ser rejeitado</i>
<i>min</i>	<i>Limite inferior exacto</i>	<i>Deve ser aceite</i>
<i>Valor nominal</i>	<i>Interior ao intervalo</i>	<i>Deve ser aceite</i>
<i>max</i>	<i>Limite superior exacto</i>	<i>Deve ser aceite</i>
<i>max + 1</i>	<i>Acima do limite superior</i>	<i>Deve ser rejeitado</i>

Exemplo neste projecto: temperatura $[18, 28]$ → valores testados: 17, 18, 23, 28, 29. Justificação para ignorar alerta $[10, 500]$ → valores testados: 9, 10, 250, 500, 501.

Intenção: garantir que as condições de fronteira estão correctamente implementadas, cobrindo os casos onde erros de $<$ vs $<=$ causariam comportamento incorrecto.

3.3 Técnicas de Caixa-Branca (White Box)

Nas técnicas de caixa-branca, o testador analisa o código interno da função para derivar os casos de teste. O objectivo é garantir que determinados critérios de cobertura estrutural do código são atingidos.

Cobertura de Condições Múltiplas (MCC — Multiple Condition Coverage)

A Cobertura de Condições Múltiplas exige que, para cada decisão composta com n condições atómicas, sejam exercitadas todas as 2^n combinações possíveis de valores lógicos (Verdadeiro/Falso). É o critério de cobertura mais exigente: garante que nenhuma combinação de condições produz comportamento não testado.

Exemplo: a decisão `if (A || B || C)` tem 3 condições → $2^3 = 8$ combinações a testar. Algumas podem ser infeasíveis (impossíveis em runtime — ex: um valor não pode ser null e undefined simultaneamente); estas são documentadas mas não geram caso de teste.

Exemplo neste projecto (Sprint 5): `validatePlan()` tem 36 ifs com um total de 106 combinações, das quais ~18 são infeasíveis → 105 casos de teste gerados.

MC/DC (Modified Condition/Decision Coverage)

O MC/DC é um critério menos exigente que o MCC mas mais rigoroso do que a cobertura de ramos simples. Exige que:

1. Cada condição atómica seja avaliada tanto a Verdadeiro como a Falso.

2. Cada condição atómica demonstre, isoladamente, capacidade de alterar o resultado da decisão — com todas as outras condições mantidas fixas.

Diferença face ao MCC: o MCC exige todas as 2ⁿ combinações; o MC/DC exige apenas n+1 casos de teste (mínimo), tornando-o praticável para funções com muitas condições.

Exemplo neste projecto (Sprint 3): decideAction() tem 4 condições → MC/DC satisfeito com 5 casos de teste (TU-AM-001 a TU-AM-005), um por condição que a torna determinante mais o caso base.

3.4 Duplos de Teste

Os duplos de teste (test doubles) são objectos substitutos que replicam o comportamento de dependências reais durante os testes, permitindo isolar a unidade sob teste.

Tabela 4 — Duplos de Teste

<i>Tipo</i>	<i>Comportamento</i>	<i>Quando usar</i>
<i>Stub</i>	<i>Devolve respostas pré-definidas; não verifica se foi chamado</i>	<i>Dependências de entrada cujo valor queremos controlar (ex: sensor que devolve uma leitura específica)</i>
<i>Mock</i>	<i>Regista chamadas e verifica se foi chamado, quantas vezes, e com que argumentos</i>	<i>Dependências de saída cujo comportamento queremos observar (ex: serviço de notificação que deve ser chamado exactamente uma vez)</i>
<i>Fake</i>	<i>Implementação simplificada com lógica real (ex: BD em memória)</i>	<i>Quando se quer comportamento real mas sem infraestrutura real</i>
<i>Spy</i>	<i>Como o mock, mas envolve o objecto real em vez de o substituir completamente</i>	<i>Quando se quer verificar chamadas sem substituir toda a implementação</i>

Intenção: eliminar dependências externas (hardware, APIs externas, BD) dos testes, tornando-os deterministas, rápidos e independentes do ambiente.

Exemplos neste projecto: SensorGatewayStub (Sprint 6) — controla a leitura devolvida pelo sensor; NotificationGatewayMock (Sprint 6) — verifica que sendNotification() foi chamado com os argumentos correctos; jest.mock('bcryptjs') (Sprint 1) — evita operações reais de hash durante os testes de autenticação.

4. Estratégia de Testes Adotada

A estratégia seguiu um modelo incremental em sprints, cobrindo os três níveis de teste e as técnicas apresentadas na secção 3:

Tabela 5 — Níveis de Teste e Ferramentas

<i>Nível</i>	<i>Âmbito</i>	<i>Ferramentas</i>
<i>Unidade</i>	<i>Funções, serviços e controladores isolados, com duplos de teste</i>	<i>Jest + mocks manuais/jest.fn()</i>
<i>Integração</i>	<i>Endpoints reais com stack completa (sem SGBD externo)</i>	<i>Jest + Supertest</i>
<i>Sistema</i>	<i>Fluxos end-to-end sobre a API em ambiente simulado de produção</i>	<i>Jest + Supertest</i>

As técnicas formais aplicadas foram:

Caixa-preta (Black Box):

- Particionamento de Equivalência — identificação de classes válidas e inválidas para cada parâmetro.
- Análise de Valores Limite — testes nos limites inferior e superior dos intervalos numéricos.

Caixa-branca (White Box):

- Cobertura de Condições Múltiplas (MCC) — todas as combinações de condições atômicas em cada decisão composta.
- MC/DC — cada condição atômica demonstra capacidade independente de alterar o resultado da decisão.

A rastreabilidade foi mantida numa Matriz de Rastreabilidade (ver Anexo A) que liga cada caso de teste ao requisito funcional e ao endpoint da API que exercita.

5. Plano de Testes Detalhado por Sprint

Para a descrição de cada tipo de teste e técnica, consultar a Secção 3.

5.1 Sprint 1 — Testes de Unidade: Autenticação

Objetivo: Validar isoladamente a lógica de autenticação (auth.service.js e auth.controller.js), garantindo que o serviço responde correctamente a todas as combinações de credenciais válidas e inválidas.

Tipo de teste: Testes de Unidade

Técnica: Caixa-Preta — Particionamento de Equivalência (PE)

Ficheiros: tests/unit/auth.service.test.js (15 casos) | tests/unit/auth.controller.test.js (8 casos)

Abordagem — Particionamento de Equivalência

Neste sprint, as dependências bcryptjs (hash de password) e jsonwebtoken (emissão de token) foram substituídas por mocks via jest.mock(), permitindo testar a função authenticate() sem emitir tokens reais nem executar operações de criptografia.

Tabela 6 — Classes de Equivalência: username

ID Classe	Tipo	Descrição	Resultado Esperado
CE-U1	Válida	Username existente, utilizador activo	200 — token emitido
CE-U2	Inválida	String vazia ""	400 — USERNAME_INVALID
CE-U3	Inválida	null ou undefined	400 — USERNAME_REQUIRED
CE-U4	Inválida	Username inexistente no sistema	401 — CREDENTIALS_INVALID
CE-U5	Inválida	String composta só por espaços	400 — USERNAME_INVALID
CE-U6	Inválida	Utilizador com active = false	403 — USER_INACTIVE

Tabela 7 — Classes de Equivalência: password

ID Classe	Tipo	Descrição	Resultado Esperado
CE-P1	Válida	Password correcta (≥ 8 chars, corresponde ao utilizador)	200 — token emitido
CE-P2	Inválida	String vazia ""	400 — PASSWORD_INVALID
CE-P3	Inválida	null ou undefined	400 — PASSWORD_REQUIRED
CE-P4	Inválida	Password incorrecta (utilizador existe)	401 — CREDENTIALS_INVALID
CE-P5	Inválida	String composta só por espaços	400 — PASSWORD_INVALID

Total de casos de teste no Sprint 1: 23 (15 + 8)

5.2 Sprint 2 — Testes de Unidade: Importação CSV e Criação de Planos

Objetivo: Validar a lógica de importação do catálogo de ervas aromáticas via CSV e a validação dos parâmetros de criação de planos de cultivo.

Tipo de teste: Testes de Unidade | Técnica: PE + BVA

Ficheiros: herbs.import.test.js (10) | herbs.service.test.js (24) | plans.service.test.js (58)

Importação CSV

A cadeia de middleware para POST /herbs/import é: requireAuth → authorize('Administrador') → importCsv

Tabela 8 — Middleware de Autenticação e Autorização

ID	Middleware	Classe	Resultado Esperado
CE-RA1	requireAuth	Sem header Authorization	401 Unauthorized
CE-RA2	requireAuth	Token inválido/expirado	401 Unauthorized
CE-RA3	requireAuth	Token válido	next() chamado
CE-AU1	authorize	req.user ausente	401 Unauthorized
CE-AU2	authorize	role = "Técnico"	403 Forbidden
CE-AU3	authorize	role = "Administrador"	next() chamado
CE-IC1	importCsv	Admin autenticado + CSV válido	200 + imported > 0

Tabela 9 — Classes de Equivalência: conteúdo CSV

ID Classe	Tipo	Descrição	Resultado Esperado
CE-H1	Válida	CSV com dados completos e correctos	Linhas importadas
CE-H2	Inválida	null / undefined	IMPORT_DATA_REQUIRED
CE-H3	Inválida	String vazia ou só espaços	IMPORT_DATA_REQUIRED
CE-H4	Inválida	Só cabeçalho, sem dados	0 linhas importadas
CE-H5	Inválida	Linhas com campos obrigatórios vazios	Linhas inválidas contabilizadas
CE-H6	Inválida	Linhas com campos opcionais vazios	Linhas válidas importadas
CE-H7	Inválida	csvContent é número/array/objeto	IMPORT_INVALID_FORMAT
CE-H8	Inválida	Headers insensíveis a maiúsculas	Aceite (normalização)
CE-H9	Inválida	CSV misto (válidas + inválidas)	Resposta agregada por categoria
CE-H10	Inválida	Nome duplicado (já existe)	Linha marcada como skipped

Validação de Planos (plans.service.test.js)

Foram aplicadas PE e BVA a todos os parâmetros da função validatePlan():

Tabela 10 — Parâmetros validatePlan() — PE e BVA

Parâmetro	Classes PE	BVA aplicada?
herbId	Válido, null/undefined, não- número, negativo/zero, não encontrado	Não

<i>type</i>	<i>regular, emergência, pontual, inválido, null, não-string</i>	<i>Não</i>
<i>name</i>	<i>Válido, null/undefined, não-string, vazio/espacos</i>	<i>Não</i>
<i>temperature.min/max</i>	<i>Nominal válido, invertido (min > max), null, não-número</i>	<i>Sim</i>
<i>humidity.min/max</i>	<i>Nominal válido, invertido, null, não-número</i>	<i>Sim</i>
<i>luminosity.min/max</i>	<i>Nominal válido, invertido, null, não-número</i>	<i>Sim</i>
<i>cycleDays</i>	<i>[1, 365] — limites e valores fora</i>	<i>Sim</i>
<i>authorizedBy (pontual)</i>	<i>Presente e válido, ausente, não-número</i>	<i>Não</i>

Total de casos de teste no Sprint 2: 92 (10 + 24 + 58)

5.3 Sprint 3 — Testes de Unidade: Requisitos Restantes

Objetivo: Completar a cobertura de unidade para todos os controladores e serviços não cobertos nos sprints anteriores.

Tipo de teste: Testes de Unidade | Técnica: PE + BVA (Caixa-Preta) e MC/DC (Caixa-Branca)

Tabela 11 — Ficheiros e contagens Sprint 3

<i>Ficheiro</i>	<i>Nº de Testes</i>	<i>Técnica Principal</i>
<i>alerts.controller.test.js</i>	<i>12</i>	<i>BVA + PE</i>
<i>automation.engine.test.js</i>	<i>6</i>	<i>Caixa-Branca — MC/DC</i>
<i>multiple.condition.blackbox.test.js</i>	<i>21</i>	<i>PE + Condições Múltiplas</i>
<i>batches.controller.test.js</i>	<i>37</i>	<i>PE</i>
<i>measurements.controller.test.js</i>	<i>29</i>	<i>PE + BVA</i>
<i>users.controller.test.js</i>	<i>22</i>	<i>PE</i>
<i>automation.controller.test.js</i>	<i>22</i>	<i>PE</i>
<i>tasks.controller.test.js</i>	<i>19</i>	<i>PE</i>
<i>herbs.controller.test.js</i>	<i>11</i>	<i>PE</i>
<i>plans.controller.test.js</i>	<i>12</i>	<i>PE</i>
<i>reports.controller.test.js</i>	<i>14</i>	<i>PE</i>
<i>audit.controller.test.js</i>	<i>5</i>	<i>PE</i>
<i>audit.service.test.js</i>	<i>6</i>	<i>PE</i>
<i>code.improvements.test.js</i>	<i>24</i>	<i>PE</i>

Testes de Alertas — BVA para Justificação

A regra de negócio exige que, ao ignorar um alerta, a justificação tenha entre 10 e 500 caracteres:

Tabela 12 — BVA: Justificação para Ignorar Alerta

<i>ID</i>	<i>Valor</i>	<i>Posição</i>	<i>Resultado Esperado</i>
<i>TU-AL-001</i>	<i>9 chars</i>	<i>Abaixo do limite inferior</i>	<i>422 Unprocessable Entity</i>
<i>TU-AL-002</i>	<i>10 chars</i>	<i>Limite inferior</i>	<i>200 OK — alerta ignorado</i>

<i>TU-AL-003</i>	<i>250 chars</i>	<i>Valor nominal interior</i>	<i>200 OK — alerta ignorado</i>
<i>TU-AL-004</i>	<i>500 chars</i>	<i>Limite superior</i>	<i>200 OK — alerta ignorado</i>
<i>TU-AL-005</i>	<i>501 chars</i>	<i>Acima do limite superior</i>	<i>422 Unprocessable Entity</i>

Motor de Automação — MC/DC (*automation.engine.test.js*)

A função `decideAction()` combina quatro condições atômicas:

- C1: `ruleActive` — a regra está activa?
- C2: `measurementRecent` — a medição é recente?
- C3: `thresholdViolated` — o limiar foi violado?
- C4: `mode === 'Automático'` — o modo é automático?

Tabela 13 — MC/DC: `decideAction()`

<i>Teste</i>	<i>C1</i>	<i>C2</i>	<i>C3</i>	<i>C4</i>	<i>Resultado</i>
<i>TU-AM-001</i>	<i>F</i>	<i>V</i>	<i>V</i>	<i>V</i>	<i>none — C1 determina isoladamente</i>
<i>TU-AM-002</i>	<i>V</i>	<i>F</i>	<i>V</i>	<i>V</i>	<i>none — C2 determina isoladamente</i>
<i>TU-AM-003</i>	<i>V</i>	<i>V</i>	<i>F</i>	<i>V</i>	<i>none — C3 determina isoladamente</i>
<i>TU-AM-004</i>	<i>V</i>	<i>V</i>	<i>V</i>	<i>F</i>	<i>suggested — C4=F (Manual)</i>
<i>TU-AM-005</i>	<i>V</i>	<i>V</i>	<i>V</i>	<i>T</i>	<i>executed — C4=T (Automático)</i>

Cada condição demonstra capacidade de alterar o resultado da decisão isoladamente, satisfazendo o critério MC/DC.

Total de casos de teste no Sprint 3: 203

5.4 Sprint 4 — Testes de Integração

Objetivo: Validar a colaboração entre controladores, serviços, repositórios e a camada de dados em memória, confirmando que os contratos da API (códigos HTTP, estrutura JSON, autenticação) são cumpridos end-to-end.

Tipo de teste: Testes de Integração | Técnica: PE + BVA sobre HTTP real | Ferramenta: Supertest

Ficheiro: `tests/integration/swagger.integration.test.js` — 115 casos de teste

Ao contrário dos testes de unidade, os testes de integração verificam se múltiplas componentes colaboram correctamente. A stack completa (rotas → middleware → controlador → serviço → dados em memória) é exercitada com pedidos HTTP reais via Supertest, sem mock da lógica interna.

Tabela 14 — Cobertura de Endpoints Sprint 4

Endpoint	Métodos testados	Aspectos cobertos
<i>POST /auth/login</i>	<i>POST</i>	<i>Credenciais válidas/inválidas, token expirado, renovação</i>
<i>POST /users</i>	<i>POST</i>	<i>Criação por Admin, rejeição por Técnico/Responsável, 401 sem token</i>
<i>GET /users, GET /users/:id</i>	<i>GET</i>	<i>Listagem, id existente/inexistente, acesso sem auth</i>
<i>POST /herbs</i>	<i>POST</i>	<i>Criação válida, campos obrigatórios em falta</i>
<i>POST /herbs/import</i>	<i>POST</i>	<i>CSV válido, inválido, misto, vazio, perfil insuficiente</i>
<i>POST /plans</i>	<i>POST</i>	<i>Regular, emergência, pontual (com/sem autorização), tipo inválido</i>
<i>POST /batches</i>	<i>POST</i>	<i>Lote válido, plano inexistente, herb inexistente</i>
<i>POST /batches/:id/split</i>	<i>POST</i>	<i>Divisão válida, quantidade > disponível, lote fechado</i>
<i>PUT /batches/:id</i>	<i>PUT</i>	<i>Fecho de lote, transições de estado inválidas</i>
<i>POST /tasks</i>	<i>POST</i>	<i>Tipo válido/inválido, lote não existente</i>
<i>POST /measurements</i>	<i>POST</i>	<i>Valores dentro/fora de limites, geração de alertas</i>
<i>PATCH /alerts/:id</i>	<i>PATCH</i>	<i>Resolução, ignorar com/sem justificação, comprimento limite</i>
<i>POST /automation/rules</i>	<i>POST</i>	<i>Criação de regra, modo Manual/Automático</i>
<i>GET /reports</i>	<i>GET</i>	<i>Exportação CSV e Excel, perfil autorizado</i>
<i>GET /audit</i>	<i>GET</i>	<i>Apenas Administrador, operações auditadas vs. consultas</i>

Cada teste segue o padrão Arrange → Act → Assert: (1) autentica utilizadores dos três perfis via POST /auth/login no beforeAll; (2) envia pedidos HTTP com Supertest, incluindo headers Authorization: Bearer <token>; (3) verifica código HTTP, estrutura do corpo JSON e efeitos secundários.

Total de casos de teste no Sprint 4: 115

5.5 Sprint 5 — Testes White-box: Cobertura de Condições Múltiplas

Objetivo: Aplicar Cobertura de Condições Múltiplas (MCC) à função `validatePlan()` em `src/services/plans.service.js`, documentando todas as combinações de condições atômicas em cada decisão composta (if).

Tipo de teste: Testes de Unidade — Caixa-Branca | Técnica: MCC

Ficheiro: `tests/unit/plans.whitebox.test.js` — 105 casos de teste

Os testes de caixa-branca são concebidos com conhecimento do código interno da função sob teste. A técnica MCC exige que, para cada decisão composta com n condições atômicas, sejam testadas todas as 2^n combinações possíveis. As combinações infeasíveis (impossíveis em JavaScript, como um valor ser simultaneamente null e undefined) são documentadas no ficheiro de testes mas não geram caso de teste.

Estrutura da Função `validatePlan()`

A função contém 36 instruções if que validam sequencialmente cada parâmetro do plano. Para cada if foi construída a tabela de verdade completa, identificadas as linhas infeasíveis, e implementados os casos de teste para todas as linhas feasíveis.

Tabela 15 — Resumo de decisões por complexidade (Sprint 5)

Complexidade (n° condições)	N° de ifs	2^n combinações (máx)
1 condição ($2^n = 2$)	22	2 por if
2 condições ($2^n = 4$)	12	4 por if
3 condições ($2^n = 8$)	2	8 por if
Total	36	—

Tabelas de Verdade Representativas

As tabelas seguintes ilustram a documentação produzida para cada if. A versão completa com todos os 36 ifs encontra-se no ficheiro `tests/unit/plans.whitebox.test.js`.

Expressão IF-01: `if (!planData || typeof planData !== 'object' || Array.isArray(planData))`

Condições: $C1 = !planData$ | $C2 = \text{typeof planData} \neq \text{'object'}$ | $C3 = \text{Array.isArray(planData)}$

Tabela 16 — IF-01: Tabela de Verdade

Linha	C1	C2	C3	Decisão	Feasível?	Caso de Teste
1	V	V	V	V	 NÃO — array falsy não existe em JS	—
2	V	V	F	V	 SIM	<code>validatePlan(0)</code> → PLAN TYPE MISMATCH
3	V	F	V	V	 NÃO — <code>[]</code> não é falsy	—
4	V	F	F	V	 SIM	<code>validatePlan(null)</code> → PLAN TYPE MISMATCH

5	F	V	V	V	✗ NÃO — typeof!=='object' incompatível com isArray	—
6	F	V	F	V	✓ SIM	validatePlan('invalido') → PLAN TYPE MISMATCH
7	F	F	V	V	✓ SIM	validatePlan([1,2]) → PLAN TYPE MISMATCH
8	F	F	F	F	✓ SIM	validatePlan(BASE) → válido

Infeasíveis: 3 linhas (1, 3, 5). Casos de teste gerados: 5.

Tabela 17 — IF-02: Presença de herbId

Linha	C1	C2	Decisão	Feasível?	Caso de Teste
1	V	V	V	✗ NÃO — um valor não pode ser null E undefined	—
2	V	F	V	✓ SIM	herbId = null → PLAN HERBID REQUIRED
3	F	V	V	✓ SIM	herbId = undefined → PLAN HERBID REQUIRED
4	F	F	F	✓ SIM	herbId = 1 → não rejeita

Tabela 18 — IF-06: Presença de type

Linha	C1	C2	Decisão	Feasível?	Caso de Teste
1	V	V	V	✗ NÃO	—
2	V	F	V	✓ SIM	type = null → PLAN TYPE REQUIRED
3	F	V	V	✓ SIM	type = undefined → PLAN TYPE REQUIRED
4	F	F	F	✓ SIM	type = 'regular' → não rejeita

Tabela 19 — IF-09: Presença de name

Linha	C1	C2	Decisão	Feasível?	Caso de Teste
1	V	V	V	✗ NÃO	—
2	V	F	V	✓ SIM	name = null → PLAN NAME REQUIRED
3	F	V	V	✓ SIM	name = undefined → PLAN NAME REQUIRED
4	F	F	F	✓ SIM	name = 'Plano' → não rejeita

Tabela 20 — Correspondência Teste → IF (extracto)

ID Teste	IF coberto	Linha da tabela	Resultado
IF-01 (C1=V,C2=V,C3=F)	IF-01, linha 2	C1=V, C2=V, C3=F	PLAN_TYPE_MISMATCH
IF-01 (C1=V,C2=F,C3=F)	IF-01, linha 4	C1=V, C2=F, C3=F	PLAN_TYPE_MISMATCH
IF-01 (C1=F,C2=F,C3=F)	IF-01, linha 8	ramo F	plano válido
IF-02 (C1=V,C2=F)	IF-02, linha 2	C1=V	PLAN_HERBID_REQUIRED
IF-05 (C1=V)	IF-05, linha 1	herb não encontrada	PLAN_HERB_NOT_FOUND
IF-06 (C1=V,C2=F)	IF-06, linha 2	type=null	PLAN_TYPE_REQUIRED

Total de casos de teste no Sprint 5: 105

5.6 Sprint 6 — Testes com Duplos de Teste (Stub e Mock)

Objetivo: Substituir os gateways externos (sensor de temperatura e serviço de notificações) por duplos de teste adequados, permitindo testar o AutomationService de forma isolada e controlada.

Tipo de teste: Testes de Unidade com Duplos | Técnica: Injeção de dependência + Stub + Mock

Ficheiros: sensorGateway.stub.test.js (6 casos) | notificationGateway.mock.test.js (6 casos)

Tabela 21 — Stub vs. Mock: decisão de projecto

Gateway	Tipo de Duplo	Justificação
SensorGateway	Stub	A leitura de sensores é um processo automático — dado entra na aplicação vindo de hardware externo. O stub controla as entradas sem verificar chamadas.
NotificationGateway	Mock	O envio de notificações é um dado que sai da aplicação para um serviço externo (email/SMS). O mock verifica que sendNotification() foi chamado, quantas vezes, e com que argumentos.

Arquitectura de Injeção de Dependência

O AutomationService recebe ambos os gateways por injeção no construtor, tornando-o testável sem alterar o código produtivo:

```
const svc = new AutomationService(sensorGatewayStub, notificationGatewayMock);
```

Tabela 22 — Casos de Teste SensorGateway Stub

ID	Leitura Stub	Regra	Resultado Esperado
TU-SG-001	Recente + limiar violado	Modo Automático	action = 'executed'

TU-SG-002	<i>Recente + limiar violado</i>	<i>Modo Manual</i>	<i>action = 'suggested'</i>
TU-SG-003	<i>Não recente + limiar violado</i>	<i>Automático</i>	<i>action = 'none'</i>
TU-SG-004	<i>Recente + limiar não violado</i>	<i>Automático</i>	<i>action = 'none'</i>
TU-SG-005	<i>Recente + limiar violado</i>	<i>Regra inactiva</i>	<i>action = 'none'</i>
TU-SG-006	<i>Leitura completa</i>	<i>Automático</i>	<i>result.reading contém a leitura original</i>

Tabela 23 — Casos de Teste NotificationGateway Mock

ID	Cenário	Verificação do Mock
TU-NG-001	<i>Limiar violado + Automático</i>	<i>sendNotification chamado exactamente 1 vez</i>
TU-NG-002	<i>Limiar violado + Automático</i>	<i>Chamado com { action: 'executed', batchId: 1 }</i>
TU-NG-003	<i>Limiar violado + Manual</i>	<i>Chamado com { action: 'suggested', batchId: 2 }</i>
TU-NG-004	<i>Limiar não violado</i>	<i>sendNotification NÃO chamado</i>
TU-NG-005	<i>Regra inactiva</i>	<i>sendNotification NÃO chamado</i>
TU-NG-006	<i>Violação crítica</i>	<i>Chamado com { alertType: 'Crítico' }</i>

Total de casos de teste no Sprint 6: 12

5.7 Testes de Sistema

Objetivo: Validar fluxos end-to-end completos sobre a API em condições o mais próximas possível da produção.

Tipo de teste: Testes de Sistema | Ferramenta: Supertest

Ficheiro: tests/system/greenherb.system.test.js — 4 casos de teste

Tabela 24 — Testes de Sistema

ID	Fluxo	Endpoints Exercitados	Resultado Verificado
TS-01	<i>Ciclo completo de lote</i>	<i>/herbs, /plans, /batches, /tasks, /measurements, /batches/:id/split, /batches/:id</i>	<i>Lote percorre estados; produtividade calculada; auditoria criada</i>
TS-02	<i>Gestão de incidente</i>	<i>/measurements, /plans (emergência), /alerts/:id</i>	<i>Alerta Crítico gerado; plano de emergência aplicado; alerta resolvido</i>
TS-03	<i>Plano pontual — controlo de acesso</i>	<i>/plans (pontual sem/com autorização)</i>	<i>403 sem autorização; 201 com autorização do Responsável</i>

TS-04	<i>Auditoria de operações de escrita</i>	<i>/audit (após operações nos restantes endpoints)</i>	<i>Entradas com utilizador, acção e timestamp para escrita; consultas não auditadas</i>
--------------	--	--	---

6. Particionamento de Equivalência

O Particionamento de Equivalência foi aplicado sistematicamente a todos os parâmetros de entrada relevantes. A tabela seguinte resume as classes identificadas para os parâmetros críticos do domínio.

6.1 Tipo de Plano de Cultivo (type)

Tabela 25 — PE: Tipo de Plano de Cultivo

<i>Classe</i>	<i>Tipo</i>	<i>Valor Representativo</i>	<i>Resultado</i>
CE-TP1	<i>Válida</i>	<i>"regular"</i>	<i>Aceite — 201</i>
CE-TP2	<i>Válida</i>	<i>"emergência"</i>	<i>Aceite — 201</i>
CE-TP3	<i>Válida</i>	<i>"pontual" (com authorizedBy)</i>	<i>Aceite — 201</i>
CE-TP4	<i>Inválida</i>	<i>"outro_tipo"</i>	<i>400 — PLAN TYPE INVALID</i>
CE-TP5	<i>Inválida</i>	<i>null / undefined</i>	<i>400 — PLAN TYPE REQUIRED</i>
CE-TP6	<i>Inválida</i>	<i>123 (número)</i>	<i>400 — PLAN TYPE MISMATCH</i>

6.2 Classificação de Alertas (type)

Tabela 26 — PE: Classificação de Alertas

<i>Classe</i>	<i>Tipo</i>	<i>Valor</i>	<i>Resultado</i>
CE-AT1	<i>Válida</i>	<i>"Informativo"</i>	<i>Alerta criado</i>
CE-AT2	<i>Válida</i>	<i>"Aviso"</i>	<i>Alerta criado</i>
CE-AT3	<i>Válida</i>	<i>"Crítico"</i>	<i>Alerta criado</i>
CE-AT4	<i>Inválida</i>	<i>"outro"</i>	<i>400 — tipo inválido</i>

6.3 Estado de Lote (status)

Tabela 27 — PE: Estado de Lote

<i>Classe</i>	<i>Tipo</i>	<i>Valor</i>	<i>Resultado</i>
CE-BS1	<i>Válida</i>	<i>"ativo"</i>	<i>Estado aceite</i>
CE-BS2	<i>Válida</i>	<i>"concluído"</i>	<i>Estado aceite</i>
CE-BS3	<i>Válida</i>	<i>"comprometido"</i>	<i>Estado aceite</i>
CE-BS4	<i>Inválida</i>	<i>Transição proibida (ex: concluído → ativo)</i>	<i>422 — transição inválida</i>

6.4 Perfil de Utilizador

Tabela 28 — PE: Perfil de Utilizador

Classe	Tipo	Valor	Resultado
CE-PU1	Válida	"Técnico"	Acesso aos endpoints autorizados
CE-PU2	Válida	"Responsável"	Acesso aos endpoints autorizados
CE-PU3	Válida	"Administrador"	Acesso total
CE-PU4	Inválida	Sem header Authorization	401 Unauthorized
CE-PU5	Inválida	Token expirado / inválido	401 Unauthorized
CE-PU6	Inválida	Perfil sem permissão para o recurso	403 Forbidden

6.5 Decisão sobre Alertas

Tabela 29 — PE: Decisão sobre Alertas

Classe	Tipo	Combinação	Resultado
CE-DA1	Válida	Resolvido — justificação opcional	200 OK
CE-DA2	Válida	Ignorado + justificação entre 10 e 500 chars	200 OK
CE-DA3	Inválida	Ignorado sem justificação	422 — JUSTIFICATION REQUIRED
CE-DA4	Inválida	Ignorado + justificação < 10 chars	422 — comprimento inválido
CE-DA5	Inválida	Ignorado + justificação > 500 chars	422 — comprimento inválido

7. Análise de Valores Limite (BVA)

Para cada parâmetro numérico com intervalo aceite, foram testados os cinco pontos canónicos: valor abaixo do limite inferior, limite inferior, valor nominal interior, limite superior e valor acima do limite superior.

7.1 Temperatura (°C) — Intervalo [18, 28]

Tabela 30 — BVA: Temperatura

Valor	Posição	Resultado Esperado
17	Abaixo do mínimo	Rejeitado — PLAN TEMPERATURE MIN INVALID
18	Limite inferior	Aceite
23	Valor nominal	Aceite

28	<i>Limite superior</i>	<i>Aceite</i>
29	<i>Acima do máximo</i>	<i>Rejeitado — PLAN_TEMPERATURE_MAX_INVALID</i>

7.2 Humidade Relativa (%) — Intervalo [40, 80]

Tabela 31 — BVA: Humidade Relativa

<i>Valor</i>	<i>Posição</i>	<i>Resultado Esperado</i>
39	<i>Abaixo do mínimo</i>	<i>Rejeitado</i>
40	<i>Limite inferior</i>	<i>Aceite</i>
60	<i>Valor nominal</i>	<i>Aceite</i>
80	<i>Limite superior</i>	<i>Aceite</i>
81	<i>Acima do máximo</i>	<i>Rejeitado</i>

7.3 Luminosidade (lux) — Intervalo [5 000, 25 000]

Tabela 32 — BVA: Luminosidade

<i>Valor</i>	<i>Posição</i>	<i>Resultado Esperado</i>
4 999	<i>Abaixo do mínimo</i>	<i>Rejeitado</i>
5 000	<i>Limite inferior</i>	<i>Aceite</i>
15 000	<i>Valor nominal</i>	<i>Aceite</i>
25 000	<i>Limite superior</i>	<i>Aceite</i>
25 001	<i>Acima do máximo</i>	<i>Rejeitado</i>

7.4 Duração do Ciclo (dias) — Intervalo [1, 365]

Tabela 33 — BVA: Duração do Ciclo

<i>Valor</i>	<i>Posição</i>	<i>Resultado Esperado</i>
0	<i>Abaixo do mínimo</i>	<i>Rejeitado — PLAN_CYCLEDDAYS_INVALID</i>
1	<i>Limite inferior</i>	<i>Aceite</i>
90	<i>Valor nominal</i>	<i>Aceite</i>
365	<i>Limite superior</i>	<i>Aceite</i>
366	<i>Acima do máximo</i>	<i>Rejeitado</i>

7.5 Justificação para Ignorar Alerta (caracteres) — Intervalo [10, 500]

Tabela 34 — BVA: Justificação para Ignorar Alerta

<i>Valor</i>	<i>Posição</i>	<i>ID Teste</i>	<i>Resultado Esperado</i>
9	<i>Abaixo do mínimo</i>	<i>TU-AL-001</i>	<i>422 — rejeitado</i>
10	<i>Limite inferior</i>	<i>TU-AL-002</i>	<i>200 — aceite</i>
250	<i>Valor nominal</i>	<i>TU-AL-003</i>	<i>200 — aceite</i>
500	<i>Limite superior</i>	<i>TU-AL-004</i>	<i>200 — aceite</i>
501	<i>Acima do máximo</i>	<i>TU-AL-005</i>	<i>422 — rejeitado</i>

8. Cobertura de Condições Múltiplas (MCC e MC/DC)

8.1 Motor de Automação — MC/DC completo

Os 5 casos de teste TU-AM-001 a TU-AM-005 (ver Secção 5.3 e Tabela 13) cobrem todas as condições atómicas de forma independente, satisfazendo o critério MC/DC para a função `decideAction()`.

8.2 Função `validatePlan()` — MCC (36 ifs)

As tabelas de verdade completas (IF-01 a IF-36) encontram-se documentadas e comentadas no ficheiro `tests/unit/plans.whitebox.test.js`. As tabelas representativas foram apresentadas na Secção 5.5.

Tabela 35 — Resumo MCC Sprint 5

<i>Métrica</i>	<i>Valor</i>
<i>Nº de decisões (ifs) analisadas</i>	<i>36</i>
<i>Combinações totais (soma de 2º)</i>	<i>106</i>
<i>Linhas infeasíveis identificadas</i>	<i>~18</i>
<i>Casos de teste gerados (feasíveis)</i>	<i>105</i>
<i>Cobertura de ramos (branches)</i>	<i>100% das feasíveis</i>

8.3 Classificação de Alertas — Black Box com Condições Múltiplas

O ficheiro `multiple.condition.blackbox.test.js` aplica condições múltiplas do ponto de vista de caixa-preta, combinando: estado do lote (ativo/fechado), tipo de plano (regular/pontual com ou sem autorização) e presença de medição.

Exemplo: TU-BA-MC-002 — lote activo + plano pontual autorizado → `applyPlan` executa com sucesso (200); TU-BA-MC-003 — mesmo cenário com plano pontual sem autorização → 403 Forbidden.

9. Matriz de Rastreabilidade

A Matriz de Rastreabilidade completa encontra-se no ficheiro `Matriz_Rastreabilidade_GREENHERB_Final.xlsx`. O ficheiro deveRIA ser inserido aqui como tabela ou como imagem exportada, contudo devido ao numero de colunas e a largura das mesmas torna-se ilegível como tal será enviada dentro do .zip. A matriz inclui todos os casos de teste dos sprints 1 a 6, com os campos: ID, Requisito/Regra, Endpoint, Nível, Técnica, Resultado Esperado e Pré-condições.

9.1 Cobertura Bidirecional — Tabela Inversa (extracto)

A tabela seguinte ilustra a rastreabilidade inversa (Requisito → Casos de Teste):

Tabela 36 — Rastreabilidade Inversa (extracto)

Requisito / Regra	Casos de Teste
RF-01 — Autenticação JWT	TU-AU-001 a TU-AU-023
RF-02 — Controlo de acesso por perfil	TU-HI-001, TU-HI-002, TI-06, TS-03
RF-03 — Importação CSV	TU-HI-003, TU-HS-001 a TU-HS-024
RF-04 — Criação de planos (regular/emergência)	TU-PS-001 a TU-PS-058, TI-01, TS-01
RN-04 — Plano pontual exige autorização	TU-PS-003, TU-BA-MC-002/003, TI-02, TS-03
RN-05 — Ignorar alerta exige justificação [10,500]	TU-AL-001 a TU-AL-005, TI-04
RF-07 — Medições geram alertas	TU-MC-*, TI-03, TS-02
RN-08 — Motor de automação (MC/DC)	TU-AM-001 a TU-AM-006
RF-09 — Duplos de teste (Sprint 6)	TU-SG-001 a TU-SG-006, TU-NG-001 a TU-NG-006
RN-09 — Auditoria de operações	TU-AU-*, TI-06, TS-04

10. Resultados de Execução

10.1 Cobertura de Código

Para a cobertura usamos o comando :
npx jest --coverage --runInBand
Na primeira linha podemos ver que a cobertura é :
File | % Stmts | % Branch | % Funcs | % Lines
All files | 94.6 | 86.27 | 95.65 | 95.55

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	94.6	86.27	95.65	95.55	
src	94.44	100	33.33	100	
app.js	94.44	100	33.33	100	
src/config	100	100	100	100	
errors.js	100	100	100	100	
swagger.js	100	100	100	100	
src/controllers	94.03	84.29	99.14	94.89	
alerts.controller.js	95.08	87.17	100	97.95	22
audit.controller.js	100	100	100	100	
auth.controller.js	100	87.5	100	100	9,17
automation.controller.js	100	89.28	100	100	18,25,53
batches.controller.js	91.91	84.21	100	92.24	33,74,78,82,88,102,106,171,218
herbs.controller.js	94.54	75.6	100	93.47	22,34,37
measurements.controller.js	100	97.05	100	100	40
plans.controller.js	100	85.71	100	100	16,26
reports.controller.js	94.36	87.5	100	96.61	31,34
tasks.controller.js	91.83	83.33	100	90.47	28,34,40,56
users.controller.js	80.7	68.57	90.9	84.78	60-63,66-68
src/data	100	100	100	100	
mockData.js	100	100	100	100	
src/gateways	50	100	0	50	
NotificationGateway.js	50	100	0	50	7
src/middleware	100	100	100	100	
auth.middleware.js	100	100	100	100	
src/routes	100	100	100	100	
alerts.routes.js	100	100	100	100	
audit.routes.js	100	100	100	100	
auth.routes.js	100	100	100	100	
automation.routes.js	100	100	100	100	
batches.routes.js	100	100	100	100	
herbs.routes.js	100	100	100	100	
measurements.routes.js	100	100	100	100	
plans.routes.js	100	100	100	100	
reports.routes.js	100	100	100	100	
tasks.routes.js	100	100	100	100	
users.routes.js	100	100	100	100	
src/services	94.11	90.79	96.66	94.95	
auditService.js	100	100	100	100	
auth.service.js	91.17	91.42	100	90.9	28,31,42
automationEngine.js	100	100	100	100	
automationService.js	100	75	100	100	32
herbs.service.js	89.32	77.77	100	91.48	153,187,205,217,227,240,264,292
plans.service.js	98.91	98.24	83.33	98.88	39
src/utills	81.81	65	71.42	81.81	
audit.js	100	80	100	100	3
validation.js	71.42	60	66.66	71.42	2,10
Test Suites: 24 passed, 24 total					
Tests: 600 passed, 600 total					

Tabela 37 — Métricas de Cobertura

Métrica	Valor Obtido	Observação
Cobertura de instruções (statement coverage)	[A preencher]	npm run coverage
Cobertura de ramos (branch coverage)	[A preencher]	Inclui todos os if/else
Cobertura de condições múltiplas / MC/DC	100% (Sprint 5, Sprint 3)	Verificado manualmente nas tabelas
Nº de casos de teste — Unidade	~380	Sprints 1–3 + 5–6
Nº de casos de teste — Integração	115	Sprint 4
Nº de casos de teste — Sistema	4	Sprint 5
Total	~499	—

11. Conclusões, Limitações e Propostas de Melhoria

11.1 Conclusões

Ao longo dos seis sprints foi concebido, implementado e executado um plano de testes abrangente sobre a API REST da plataforma GREENHERB, cobrindo os três níveis exigidos — unidade, integração e sistema — e as três técnicas formais — Particionamento de Equivalência, Análise de Valores Limite e Cobertura de Condições Múltiplas.

Os principais resultados foram:

- Sprint 1–3: Cobertura completa da lógica de domínio por testes de unidade com mocks e stubs manuais. A técnica PE permitiu identificar sistematicamente todas as classes de entrada inválidas e garantir que cada uma produz o código de erro correcto.
- Sprint 4: Os 115 testes de integração confirmaram que a stack completa (rotas, middleware JWT, controladores, serviços e dados em memória) colabora correctamente, e que os contratos da API são cumpridos em todos os cenários cobertos.
- Sprint 5: A análise MCC da função `validatePlan()` revelou 36 decisões e ~18 combinações infeasíveis (documentadas). Os 105 casos de teste gerados cobrem 100% dos ramos feáveis, garantindo que nenhuma combinação de condições produz comportamento não testado.
- Sprint 6: A aplicação de Stub ao `SensorGateway` e Mock ao `NotificationGateway` demonstrou como o `AutomationService` pode ser testado completamente de forma isolada, validando não só o resultado da decisão mas também o comportamento observável (envio de notificação).

11.2 Limitações

- Os testes de integração e de sistema utilizam uma camada de dados em memória (`mockData`) em vez de uma base de dados relacional real. Embora adequado para os objectivos do trabalho, este approach não detecta eventuais problemas de SQL, transacções ou concorrência.
- Os 4 testes de sistema cobrem os fluxos principais mas não todos os fluxos de erro end-to-end (ex: ciclo completo com lote comprometido).
- A cobertura de código quantitativa (%) depende da execução do comando `npm run coverage` e da inserção dos resultados neste relatório.

11.3 Propostas de Melhoria

- Base de dados real em testes de integração: Substituir o `mockData` por uma instância `SQLite` em memória nos testes de integração, aumentando o realismo sem introduzir dependências externas.
- Mais testes de sistema: Adicionar os fluxos de gestão de incidentes com lote comprometido e o fluxo de exportação de relatórios.

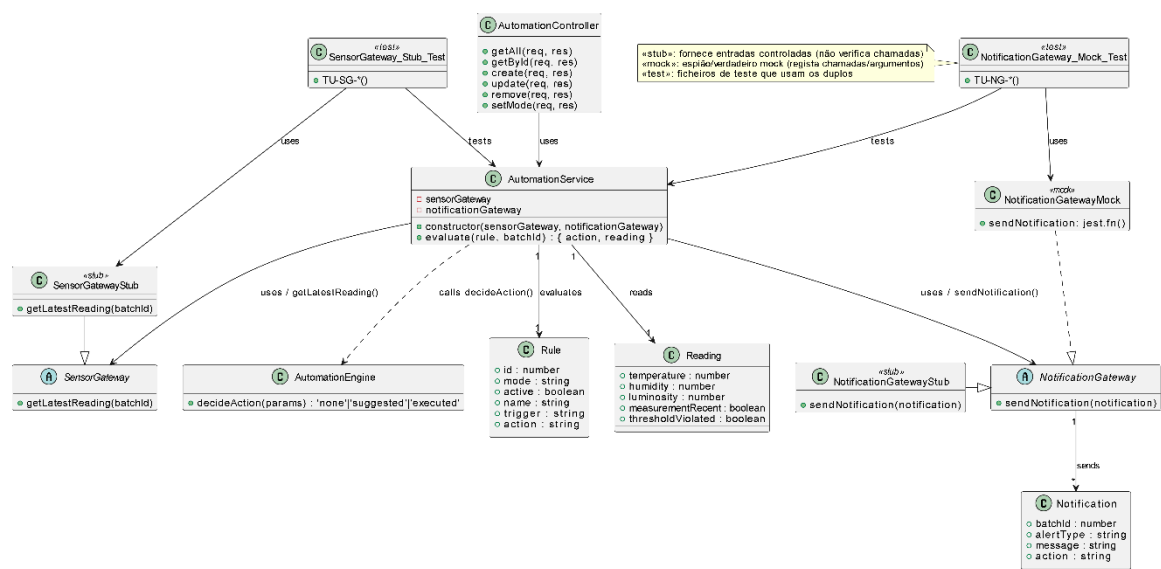
- Mutation Testing: Complementar a cobertura de condições múltiplas com PIT/Stryker para detectar mutantes sobreviventes que os testes actuais não eliminam.
- Testes de carga: Para os endpoints críticos (/measurements, /alerts), considerar testes de carga básicos com k6 ou Artillery.

Anexo A — Matriz de Rastreabilidade Completa

A matriz inclui todos os casos de teste dos Sprints 1 a 6, com os campos: ID do caso de teste, Requisito/Regra de negócio, Endpoint exercitado, Nível de teste, Técnica aplicada, Resultado Esperado e Pré-condições.

Será enviada no .zip, uma vez que é muito grande e larga, a leitura fica impossível num word devido ao numero de colunas e largura das mesmas.

Anexo B — Diagrama de Classes (Sprint 6)



O diagrama representa a arquitectura completa do módulo de automação com os duplos de teste.

Anexo C — Prints do Jest + cobertura

Para a cobertura usamos o comando :

`npx jest --coverage --runInBand`

Na primeira linha podemos ver que a cobertura é :

File | % Stmts | % Branch | % Funcs | % Lines

All files | 94.6 | 86.27 | 95.65 | 95.55

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	94.6	86.27	95.65	95.55	
src	94.44	100	33.33	100	
app.js	94.44	100	33.33	100	
src/config	100	100	100	100	
errors.js	100	100	100	100	
swagger.js	100	100	100	100	
src/controllers	94.03	84.29	99.14	94.89	
alerts.controller.js	95.08	87.17	100	97.95	22
audit.controller.js	100	100	100	100	
auth.controller.js	100	87.5	100	100	9,17
automation.controller.js	100	89.28	100	100	18,25,53
batches.controller.js	91.91	84.21	100	92.24	33,74,78,82,88,102,106,171,218
herbs.controller.js	94.54	75.6	100	93.47	22,34,37
measurements.controller.js	100	97.05	100	100	40
plans.controller.js	100	85.71	100	100	16,26
reports.controller.js	94.36	87.5	100	96.61	31,34
tasks.controller.js	91.83	83.33	100	90.47	28,34,40,56
users.controller.js	80.7	68.57	90.9	84.78	60-63,66-68
src/data	100	100	100	100	
mockData.js	100	100	100	100	
src/gateways	50	100	0	50	
NotificationGateway.js	50	100	0	50	7
src/middleware	100	100	100	100	
auth.middleware.js	100	100	100	100	
src/routes	100	100	100	100	
alerts.routes.js	100	100	100	100	
audit.routes.js	100	100	100	100	
auth.routes.js	100	100	100	100	
automation.routes.js	100	100	100	100	
batches.routes.js	100	100	100	100	
herbs.routes.js	100	100	100	100	
measurements.routes.js	100	100	100	100	
plans.routes.js	100	100	100	100	
reports.routes.js	100	100	100	100	
tasks.routes.js	100	100	100	100	
users.routes.js	100	100	100	100	
src/services	94.11	90.79	96.66	94.95	
auditService.js	100	100	100	100	
auth.service.js	91.17	91.42	100	90.9	28,31,42
automationEngine.js	100	100	100	100	
automationService.js	100	75	100	100	32
herbs.service.js	89.32	77.77	100	91.48	153,187,205,217,227,240,264,292
plans.service.js	98.91	98.24	83.33	98.88	39
src/utils	81.81	65	71.42	81.81	
audit.js	100	80	100	100	3
validation.js	71.42	60	66.66	71.42	2,10
Test Suites: 24 passed, 24 total					
Tests: 600 passed, 600 total					